

Programming Exercise 03: Recursion and Dynamic Programming

Data Structure and Database
Shiraz University

January 2, 2026

1 Decimal To Binary

Write a recursive Python function called `decimal_to_binary(n)` that takes a non-negative integer n (in base 10) and returns its binary representation as a string (in base 2), without using built-in functions like `bin()` or string formatting.

Your solution must use recursion—no loops allowed!

2 Gray Codes

A **Gray code** is a binary numeral system where two consecutive values differ in only one bit. For example, the 2-bit Gray code is:

`["00", "01", "11", "10"]`

Notice that each adjacent pair ($00 \rightarrow 01$, $01 \rightarrow 11$, $11 \rightarrow 10$) flips just one bit.

The n -bit Gray code can be constructed recursively from the $(n - 1)$ -bit sequence:

1. Take the $(n - 1)$ -bit Gray code and prepend '0' to each string.
2. Take the $(n - 1)$ -bit Gray code in **reverse order** and prepend '1' to each string.
3. Concatenate the two lists.

Base Cases:

4. $n = 0 \rightarrow$ return `[""]`
5. $n = 1 \rightarrow$ return `['0', '1']`

2.1 Task

Write a Python function `gray_code(n)` that returns a list of strings representing the n -bit Gray code sequence, constructed **recursively**.

Function Signature:

```
def gray_code(n: int) -> list[str]:  
    pass
```

3 Pretty Printing

In modern word processors and typesetting systems like L^AT_EX or T_EX, a common task is to reformat a block of text so that it looks visually pleasing. This is known as the *pretty printing* problem. The goal is to break a sequence of words into lines such that the right margin appears as “even” as possible—without hyphenating words or inserting extra inter-word spaces.

3.1 The Goal

Given a paragraph of text and a maximum line width, we want to split the text into lines that look "even" or "neat." For example:

Ugly formatting (ragged):
This is an
example of text that
has been formatted
poorly.

Better formatting (even):
This is an example of
text that has been
formatted well with
more even line lengths.

3.2 The Rules

1. Each line can hold at most L characters (including spaces)
2. We put exactly one space between words on a line
3. We cannot break words across lines (no hyphenation)
4. The "slack" of a line is the number of unused spaces at the end

3.3 Example

Suppose we have the words: ["The", "quick", "brown", "fox"] and $L = 10$ characters.

Option 1: Put all words on one line

The quick
brown fox

Line 1 has 9 characters used, slack = 1

Line 2 has 9 characters used, slack = 1

Option 2: Different arrangement

The
quick
brown fox

Line 1 has 3 characters used, slack = 7

Line 2 has 5 characters used, slack = 5

Line 3 has 9 characters used, slack = 1

Clearly Option 1 looks better! But how do we formalize "better"?

4 Formalizing the Problem

4.1 Basic Definitions

Let:

- W = array of word lengths, where $W[i]$ is the length of word i
- L = maximum line length (in characters)
- n = total number of words

4.2 Computing Line Length

If we print words i through k on a single line, the total characters used is:

$$N[i, k] = \sum_{j=i}^k W[j] + (k - i) \quad (1)$$

Why? We have $(k - i)$ spaces between words. For example, with 3 words, we need 2 spaces.

4.3 Slack Cost

The "slack" of a line is the unused space. We define:

$$\text{slack}[i, k] = L - N[i, k] \quad (2)$$

If $\text{slack}[i, k] < 0$, the words don't fit on one line, so we set the cost to ∞ .

4.4 Total Cost Functions

To measure how "even" our text looks, we sum up the slack costs across all lines. We'll explore different ways to measure this:

4.4.1 Cost Function 1: Sum of Squares (Most Common)

$$S_{\text{square}}[i, k] = \begin{cases} (L - N[i, k])^2 & \text{if } L - N[i, k] \geq 0 \\ \infty & \text{otherwise} \end{cases} \quad (3)$$

Why squares? Squaring penalizes large gaps more heavily. A line with slack 10 is worse than two lines with slack 5 each ($100 > 25 + 25$).

4.4.2 Cost Function 2: Sum of Absolute Values

$$S_{\text{abs}}[i, k] = \begin{cases} |L - N[i, k]| & \text{if } L - N[i, k] \geq 0 \\ \infty & \text{otherwise} \end{cases} \quad (4)$$

Why absolute values? This treats all slack equally. A line with slack 10 equals two lines with slack 5 each ($10 = 5 + 5$).

4.4.3 Cost Function 3: Sum of Cubes

$$S_{\text{cube}}[i, k] = \begin{cases} (L - N[i, k])^3 & \text{if } L - N[i, k] \geq 0 \\ \infty & \text{otherwise} \end{cases} \quad (5)$$

Why cubes? This penalizes large gaps even more severely than squares.

5 Dynamic Programming Solution

5.1 The Key Insight

Suppose we've found the best way to format the first i words. If we want to add one more word, we only need to reconsider the last line—the earlier lines stay the same!

This gives us the recurrence:

$$C[i] = \begin{cases} 0 & \text{if } i = 0 \\ \min_{1 \leq k \leq i} \{C[k-1] + S[k, i]\} & \text{if } i \geq 1 \end{cases} \quad (6)$$

where:

- $C[i]$ = minimum cost to format the first i words
- k = the first word on the last line
- $S[k, i]$ = cost of putting words k through i on the last line

6 Your Assignment

6.1 Part 1: Written Analysis (20 points)

Answer the following questions in 2-3 paragraphs:

1. Does the recurrence change with different cost functions?
2. Predict: which cost function (squares, absolute values, or cubes) will produce the most "even-looking" text? Why?

6.2 Part 2: Implementation (60 points)

Implement the text formatting algorithm with all three cost functions:

6.3 Part 3: Testing and Comparison (20 points)

6.3.1 Test Case 1: Simple Example

Words: ["The", "quick", "brown", "fox", "jumps", "over",
"the", "lazy", "dog"]
Word lengths: [3, 5, 5, 3, 5, 4, 3, 4, 3]
L = 16

6.3.2 Test Case 2: From the Paper (Figure 1)

Words and lengths:

"In" (2), "a" (1), "word" (4), "processor" (9), "or" (2),
"in" (2), "LaTeX" (5), "one" (3), "routinely" (9),
"encounters" (10), "the" (3), "pretty" (6), "printing" (8),
"problem" (7), "That" (4), "is" (2), "how" (3), "does" (4),
"one" (3), "transform" (9), "text" (4), "with" (4), "a" (1),
"ragged" (6), "right" (5), "margin" (6), "into" (4),
"text" (4), "whose" (5), "right" (5), "margin" (6), "is" (2),
"as" (2), "even" (4), "as" (2), "possible" (8), "This" (4),
"write-up" (8), "discusses" (9), "the" (3), "design" (6),
"and" (3), "analysis" (8)

L = 38

6.3.3 Test Case 3: Edge Case

Words: ["a", "b", "c", "d", "e", "f", "g", "h"]

Word lengths: [1, 1, 1, 1, 1, 1, 1, 1]

L = 10

6.3.4 Required Output

For each test case and each cost function, provide:

1. The formatted text showing line breaks
2. The total cost
3. The slack of each line

6.3.5 Comparative Analysis

Write 2-3 paragraphs comparing the results:

- How do the three cost functions differ in their line break decisions?
- Which produces the most visually appealing output? Why?
- Were there any surprising differences?
- Did your prediction from Part 1 match the actual results?

Good luck! Start early and ask questions if you get stuck.